Reviewer Pack — Hilbert Cube Diameter and Frege Proof Depth

Entry 045be409c40e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 07:38:13 UTC

Hilbert Cube Diameter and Frege Proof Depth

Entry ID: `045be409c40e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 07:38:13 UTC

1. Conjecture

Field A (mathematical branch): Topological Dynamics (Hilbert Cube) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

> ["For any 3CNF formula F with n variables, the diameter of the Hilbert cube representation of F's truth table is upper bounded by a function of its Frege proof depth.", 'Formally, for all 3CNF formulas F with n variables, there exists a constant c such that the diameter d(HilbertCube(F)) ≤ c * ProofDepth_Frege(F).', 'Furthermore, this bound holds with high probability over a random assignment of truth values to the variables of F.']

Rationale (proposer's reasoning):

> ['The Hilbert cube provides a topological interpretation of truth assignments, and its diameter might capture structural properties relevant to proof complexity.', 'This conjecture could expose a novel connection between topology and computational difficulty if true, leading to new insights into the nature of NP problems.']

Taxonomy category: `TOPO_FREGEPROOF` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ce0dd1e01c883736`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all 3CNF formulas F with n ≤ 40, the computed diameter d(HilbertCube(F)) is less than or equal to a constant c times the Frege proof depth ProofDepth_Frege(F) with high probability over 30 random seeds. The criterion is falsified if any seed produces a ratio of d(HilbertCube(F)) / ProofDepth_Frege(F) greater than c.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hilbert Cube diameter" AND "Frege proof depth" - "truth table representation" INtopological dynamics AND Frege proof complexity - "upper bound" ON diameter Hilbert Cube AND 3CNF formula ProofDepth_Frege

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_3cnf(n):
    clauses = []
    for _ in range(10 * n):  # Generate enough clauses to cover all variables
        clause = [random.randint(1, n) if i == 0 else -random.randint(1, n)
                  for i in range(3)]
        random.shuffle(clause)
        clauses.append(tuple(clause))
    return clauses

def truth_table(n, clauses):
    table = {}
    for assignment in product([-1, 1], repeat=n):
        table[assignment] = all(any(lit * x >= 0 for lit in clause) for clause in clauses)
    return table

def hilbert_cube_diameter(table):
    points = list(table.keys())
    n = len(points[0])
    max_dist = 0
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = sum(abs(points[i][k] - points[j][k]) for k in range(n))
            if dist > max_dist:
                max_dist = dist
    return max_dist

def frege_proof_depth(clauses):
    # Simplified DPLL-based solver to estimate proof depth
    def dpll(assignment, clauses):
        unsatisfied_clauses = [c for c in clauses if not any(lit * assignment[abs(lit) - 1] >= 0 f
```

```
        if not unsatisfied_clauses:
            return 0
        unit_clause = next((c for c in unsatisfied_clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            new_assignment = assignment[:]
            new_assignment[abs(literal) - 1] = literal // abs(literal)
            return 1 + dpll(new_assignment, [c for c in unsatisfied_clauses if literal not in c])
        pure_literal = next((l for l in range(1, len(assignment) + 1) if (l in assignment and -l n
        if pure_literal:
            new_assignment = assignment[:]
            new_assignment[pure_literal - 1] = 1
            return 1 + dpll(new_assignment, [c for c in unsatisfied_clauses if pure_literal not in
        return math.inf

    return dpll([0] * len(clauses), clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    clauses = generate_3cnf(n)
    table = truth_table(n, clauses)
    diameter = hilbert_cube_diameter(table)
    depth = frege_proof_depth(clauses)
    metric_value = diameter / depth if depth > 0 else float('inf')
    conjecture_holds = metric_value < 10 * n  # Arbitrary constant c
    counterexample = "" if conjecture_holds else f"n={n}, d={diameter}, depth={depth}"
    return {
        "metric_name": "Diameter/Depth Ratio",
        "metric_value": metric_value,
        "instances_tested": len(clauses),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    from itertools import product

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
```

```
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={supp
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2de01d7.py", line 95, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2de01d7.py", line 69, in run_trial
    table = truth_table(n, clauses)
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2de01d7.py", line 30, in truth_table
    table[assignment] = all(any(lit * x >= 0 for lit in clause) for clause in clauses)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2de01d7.py", line 30, in <genexpr>
    table[assignment] = all(any(lit * x >= 0 for lit in clause) for clause in clauses)
                            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2de01d7.py", line 30, in <genexpr>
    table[assignment] = all(any(lit * x >= 0 for lit in clause) for clause in clauses)
                                       ^
NameError: name 'x' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete the computation necessary to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11092
2	preregistration	ollama_remote	glm4:latest	0	0	5853
3	novelty	ollama_remote	glm4:latest	0	0	4682
4	novelty	ollama_remote	glm4:latest	0	0	5324
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12399
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8897
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12175
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13318
9	judge	ollama_remote	glm4:latest	0	0	11193

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 84934 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/045be409c40e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/045be409c40e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/045be409c40e.tar.gz` (if generated)